

ComputeModule: Implementing dependencies_received()

This guide covers *only* the part that still needs to be implemented: the GEMM and ALU logic inside `dependencies_received()`. Everything else (fetching, handshakes, dep-pushing) is already done.

1. Where We Are Right Now

The existing stub is:

```
void ComputeModule::dependencies_received() {
    // all dependencies received – instruction is cleared to run
    finish.notify(latency()); // ← wait N nanoseconds, then call finalize_instruction()
}
```

This is **timing-only**: it advances simulated time correctly, but does not actually perform the matrix multiply or ALU operation. The full implementation puts the computation *before* `finish.notify()`:

```
void ComputeModule::dependencies_received() {

    std::string name = current->get_name();

    if (name == "GEMM") {
        // ← GEMM implementation goes here (Section 3)
    }
    else if (name == "ALU" /* or any ALU variant name */) {
        // ← ALU implementation goes here (Section 4)
    }
    else if (name == "LOAD ACC") {
        // ← bias load into acc_mem goes here (Section 5)
    }
    // NOP and FINISH: nothing to compute – fall through to finish.notify()

    finish.notify(latency()); // ← always called last – advances simulation time
}
```

Key point: `finish.notify(latency())` is always called at the end — it doesn't matter whether we computed or not. The simulation timing must still advance.

2. Memory Arrays — What They Are and Where They Live

In the HLS code, the memory buffers are passed as function arguments. In SystemC they must be **static member variables** declared in the class — because Load, Compute, and Store all share some of them:

Array	Type	Size	Who reads it	Who writes it	HLS equivalent
uop_mem	uint32_t []	[2048]	Compute	Compute (LOAD UOP)	uop_T uop_mem[VTA_UOP_BUFF_DEPTH]
acc_mem	int32_t [] []	[512][BATCH] [BLOCK_OUT]	Compute	Compute	bus_T acc_mem[VTA_ACC_BUFF_DEPTH] [ACC_MAT_AXI_RATIO]
inp_mem	int8_t [] []	[512][BATCH] [BLOCK_IN]	Compute	LoadModule	bus_T inp_mem[VTA_INP_BUFF_DEPTH] [INP_MAT_AXI_RATIO]
wgt_mem	int8_t [] []	[512] [BLOCK_OUT] [BLOCK_IN]	Compute	LoadModule	bus_T wgt_mem[VTA_WGT_BUFF_DEPTH] [WGT_MAT_AXI_RATIO]
out_mem	int8_t [] []	[512][BATCH] [BLOCK_OUT]	StoreModule	Compute	bus_T out_mem[VTA_ACC_BUFF_DEPTH] [OUT_MAT_AXI_RATIO]

VTA default hardware sizes (from hw_spec.h):

VTA_BATCH = 1 | VTA_BLOCK_IN = 16 | VTA_BLOCK_OUT = 16

These constants mean every "tensor tile" is a 1×16 input vector and a 16×16 weight matrix.

Add these as **static members in compute_module.h** (static = one copy shared across all instances):

```
// inside class ComputeModule – new additions for real computation
static uint32_t uop_mem[2048]; // micro-op SRAM (compute-private)
static int32_t acc_mem[512][1][16]; // accumulator SRAM [depth][batch][block_out]
static int8_t inp_mem[512][1][16]; // input SRAM [depth][batch][block_in] – filled by LoadModule
static int8_t wgt_mem[512][16][16]; // weight SRAM [depth][block_out][block_in] – filled by LoadModule
static int8_t out_mem[512][1][16]; // output SRAM [depth][batch][block_out] – read by StoreModule
```

And in **compute_module.cpp** outside the class (to allocate them):

```
uint32_t ComputeModule::uop_mem[2048] = {};
int32_t ComputeModule::acc_mem[512][1][16] = {};
int8_t ComputeModule::inp_mem[512][1][16] = {};
int8_t ComputeModule::wgt_mem[512][16][16] = {};
int8_t ComputeModule::out_mem[512][1][16] = {};
```

3. The Micro-Op: What It Is and How to Decode It

Before writing GEMM/ALU, you need to understand the **micro-op (uop)**. It is a 32-bit word stored in `uop_mem[]`. It encodes the SRAM tile indices for one "elementary operation".

```
uop (32 bits):
bits [10: 0] = dst_idx_base → which accumulator tile to read/write
bits [21:11] = src_idx_base → which input tile to read (GEMM) or which acc tile to read (ALU)
bits [31:22] = wgt_idx_base → which weight tile to read (GEMM only)
```

In HLS: uses `uop.range(hi, lo)` to extract bitfields (Xilinx `ap_int` method).

In SystemC: use bitshift + bitmask — same result:

What we're extracting	HLS (vta.cc)	SystemC equivalent
dst base index (bits 10:0)	<code>uop.range(VTA_UOP_GEM_0_1, VTA_UOP_GEM_0_0)</code>	<code>(uop >> 0) & 0x7FF</code>
src base index (bits 21:11)	<code>uop.range(VTA_UOP_GEM_1_1, VTA_UOP_GEM_1_0)</code>	<code>(uop >> 11) & 0x7FF</code>
wgt base index (bits 31:22)	<code>uop.range(VTA_UOP_GEM_2_1, VTA_UOP_GEM_2_0)</code>	<code>(uop >> 22) & 0x3FF</code>

The **actual SRAM index** = base from micro-op + loop offset accumulated so far:

```
uint32_t uop = uop_mem[upc];

int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in; // accumulator tile index
int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in; // input tile index
int wgt_idx = (int)((uop >> 22) & 0x3FF) + wgt_offset_in; // weight tile index
```

4. GEMM — Full SystemC Implementation with Line-by-Line Explanation

HLS runs 3 nested loops (outer × inner × micro-op) and inside each micro-op, a 3-level tensor loop (batch × output-channel × input-channel). SystemC has no `#pragma HLS` — we just write standard C++ nested for-loops.

4.1 Instruction field → getter name mapping

HLS field name (VTAGemInsn)	Getter in SystemC Instruction*	What it means
<code>insn.iter_out</code>	<code>current->get_outer_loop_iter()</code>	How many times the outer loop runs
<code>insn.iter_in</code>	<code>current->get_inner_loop_iter()</code>	How many times the inner loop runs
<code>insn.uop_bgn</code>	<code>current->get_range_0()</code>	Micro-op start index in <code>uop_mem</code>
<code>insn.uop_end</code>	<code>current->get_range_1()</code>	Micro-op end index (exclusive)

<code>insn.dst_factor_out</code>	<code>current->get_gemm_outer_loop_acc()</code>	Stride to advance acc offset each outer iteration
<code>insn.src_factor_out</code>	<code>current->get_gemm_outer_loop_inp()</code>	Stride to advance inp offset each outer iteration
<code>insn.wgt_factor_out</code>	<code>current->get_gemm_outer_loop_wgt()</code>	Stride to advance wgt offset each outer iteration
<code>insn.dst_factor_in</code>	<code>current->get_gemm_inner_loop_acc()</code>	Stride to advance acc offset each inner iteration
<code>insn.src_factor_in</code>	<code>current->get_gemm_inner_loop_inp()</code>	Stride to advance inp offset each inner iteration
<code>insn.wgt_factor_in</code>	<code>current->get_gemm_inner_loop_wgt()</code>	Stride to advance wgt offset each inner iteration
<code>insn.reset_reg</code>	<code>current->get_reset_out()</code>	If true: zero the accumulator instead of accumulating (used for bias-free first pass)

4.2 The GEMM code, line by line

```
// — called when all dep tokens received, instruction is GEMM —————

// STEP 1: initialise outer-loop offsets to zero
// these accumulate as the loops advance, pointing to the right SRAM tiles
int dst_offset_out = 0;    // accumulator offset – grows by get_gemm_outer_loop_acc() each outer iter
int src_offset_out = 0;    // input offset      – grows by get_gemm_outer_loop_inp() each outer iter
int wgt_offset_out = 0;    // weight offset     – grows by get_gemm_outer_loop_wgt() each outer iter

// STEP 2: outer loop – iter_out iterations
// mirrors: EXE_OUT_LOOP in vta.cc gemm()
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

    // inner offsets start from where the outer offset is now
    int dst_offset_in = dst_offset_out;
    int src_offset_in = src_offset_out;
    int wgt_offset_in = wgt_offset_out;

    // STEP 3: inner loop – iter_in iterations
    // mirrors: EXE_IN_LOOP in vta.cc gemm()
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // STEP 4: micro-op loop – iterates over uop_mem[range_0 .. range_1]
        // each uop encodes one (dst_tile, src_tile, wgt_tile) triplet
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {

            // STEP 5: read the 32-bit micro-op word from uop_mem
            uint32_t uop = uop_mem[upc];

            // STEP 6: decode the three SRAM tile indices from the micro-op bitfields
            // and add the loop offsets accumulated so far
            int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in; // acc tile to write
            int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in; // inp tile to read
```

```

int wgt_idx = (int)((uop >> 22) & 0x3FF) + wgt_offset_in; // wgt tile to read

// STEP 7: triple tensor loop – batch × output-channel × input-channel
// VTA_BATCH=1, VTA_BLOCK_OUT=16, VTA_BLOCK_IN=16 by default
for (int b = 0; b < 1 /*VTA_BATCH*/; b++) {
    for (int oc = 0; oc < 16 /*VTA_BLOCK_OUT*/; oc++) {

        // STEP 8: load current accumulator value for this output element
        int32_t accum = acc_mem[dst_idx][b][oc];

        // STEP 9: dot-product over all input channels
        int32_t tmp = 0;
        for (int ic = 0; ic < 16 /*VTA_BLOCK_IN*/; ic++) {
            int8_t w_elem = wgt_mem[wgt_idx][oc][ic]; // weight element
            int8_t i_elem = inp_mem[src_idx][b][ic]; // input element
            tmp += (int32_t)(i_elem * w_elem); // multiply-accumulate (MAC)
        }

        // STEP 10: add dot-product result to the running accumulator
        accum += tmp;

        // STEP 11: write back to accumulator SRAM
        // if reset_out=true: set to 0 (used to initialise before adding bias)
        acc_mem[dst_idx][b][oc] = current->get_reset_out() ? 0 : accum;

        // STEP 12: write truncated result to output SRAM (for Store to pick up)
        // truncate 32-bit accum to 8-bit output (VTA_OUT_WIDTH = 8)
        out_mem[dst_idx][b][oc] = (int8_t)(accum & 0xFF);
    }
}

} // end micro-op loop

// STEP 13: advance inner offsets by the per-iteration strides
dst_offset_in += current->get_gemm_inner_loop_acc();
src_offset_in += current->get_gemm_inner_loop_inp();
wgt_offset_in += current->get_gemm_inner_loop_wgt();

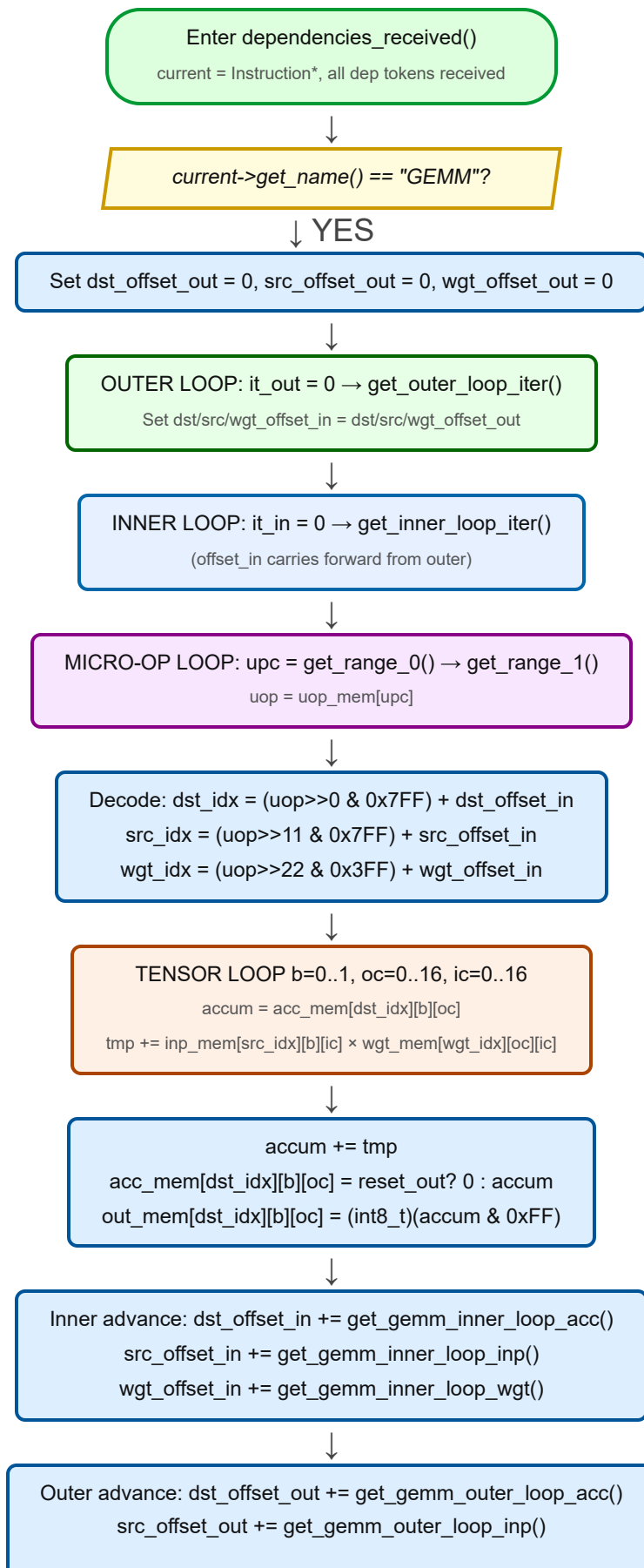
} // end inner loop

// STEP 14: advance outer offsets by the per-iteration strides
dst_offset_out += current->get_gemm_outer_loop_acc();
src_offset_out += current->get_gemm_outer_loop_inp();
wgt_offset_out += current->get_gemm_outer_loop_wgt();

} // end outer loop

```

5. GEMM Dataflow Flowchart (with actual variable names)



```
wgt_offset_out += get_gemm_outer_loop_wgt()
```



```
finish.notify(latency()) — advance simulation time by GEMM ns
```

Why three levels of loops?

The outer loop walks through different *groups* of tiles (e.g. different output feature maps). The inner loop walks different *input batches* within a group. The micro-op loop picks the exact tile triplet to compute. The tensor loop (BATCH×BLOCK_OUT×BLOCK_IN) does the actual dot product for one tile.

6. ALU — Full SystemC Implementation with Line-by-Line Explanation

ALU operates on the accumulator SRAM only — no weight memory. It applies element-wise operations: **MAX** (ReLU), **ADD**, **SHR** (shift-right). The loops are the same shape as GEMM, just no wgt dimension.

6.1 Instruction field → getter name mapping

HLS field name (VTAAluInsn)	Getter in SystemC Instruction*	What it means
<code>insn.iter_out</code>	<code>current->get_outer_loop_iter()</code>	Outer loop count
<code>insn.iter_in</code>	<code>current->get_inner_loop_iter()</code>	Inner loop count
<code>insn.uop_bgn</code>	<code>current->get_range_0()</code>	Micro-op start index
<code>insn.uop_end</code>	<code>current->get_range_1()</code>	Micro-op end index (exclusive)
<code>insn.dst_factor_out</code>	<code>current->get_alu_outer_loop_dst()</code>	acc dst offset stride (outer)
<code>insn.src_factor_out</code>	<code>current->get_alu_outer_loop_src()</code>	acc src offset stride (outer)
<code>insn.dst_factor_in</code>	<code>current->get_alu_inner_loop_dst()</code>	acc dst offset stride (inner)
<code>insn.src_factor_in</code>	<code>current->get_alu_inner_loop_src()</code>	acc src offset stride (inner)

ALU micro-op bitfield layout is different from GEMM: bits [10:0] = `dst_idx`, bits [21:11] = `src_idx`. No `wgt` field. Use `(uop >> 22)` would give garbage — just skip it for ALU.

6.2 ALU operations and the opcode

HLS ALU opcode	Value	Operation	Use in ResNet
<code>VTA_ALU_OPCODE_MIN</code>	0	<code>result = min(src_0, src_1)</code>	Rarely — clipping
<code>VTA_ALU_OPCODE_MAX</code>	1	<code>result = max(src_0, src_1)</code>	ReLU: <code>MAX(value, 0) → src_1 = imm = 0</code>
<code>VTA_ALU_OPCODE_ADD</code>	2	<code>result = src_0 + src_1</code>	Residual connections
<code>VTA_ALU_OPCODE_SHR</code>	3	<code>result = src_0 >> src_1</code>	Fixed-point quantisation scaling

ReLU explained: $\text{ReLU}(x) = \max(x, 0)$. In VTA this is: ALU with opcode MAX, `use_imm=true`, `imm=0`. So `src_1 = 0` (from immediate), and `result = max(acc_value, 0)`.

6.3 The ALU code, line by line

```
// — called when instruction is ALU (ReLU, ADD, SHR, MIN, MAX) —————

// STEP 1: initialise outer offsets
int dst_offset_out = 0; // accumulator destination offset
int src_offset_out = 0; // accumulator source offset (second operand)
```



```

// STEP 2: outer loop
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
    int dst_offset_in = dst_offset_out;
    int src_offset_in = src_offset_out;

    // STEP 3: inner loop
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // STEP 4: micro-op loop – each uop picks one (dst, src) tile pair
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
            uint32_t uop = uop_mem[upc];

            // STEP 5: decode – ALU uop has dst at [10:0], src at [21:11]
            int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in; // acc tile to write
            int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in; // acc tile to read as src

            // STEP 6: element-wise loop over the tile (batch × output-channel)
            for (int b = 0; b < 1 /*VTA_BATCH*/; b++) {
                for (int oc = 0; oc < 16 /*VTA_BLOCK_OUT*/; oc++) {

                    // STEP 7: read both operands from acc_mem
                    int32_t src_0 = acc_mem[dst_idx][b][oc]; // first operand = dst tile value
                    int32_t src_1 = acc_mem[src_idx][b][oc]; // second operand = src tile value
                    // (if use_imm=true in HLS, src_1 = insn.imm instead – e.g. 0 for ReLU)
                    // we treat src_1 from acc_mem here; for ReLU src_idx tile will contain 0s

                    // STEP 8: apply the operation
                    int32_t result;

                    // Determine opcode from instruction name:
                    // "MAX" (ReLU-like): result = max(src_0, src_1)
                    if (current->get_name() == "ALU-MAX" || current->get_name() == "ALU") {
                        result = (src_0 > src_1) ? src_0 : src_1;
                    }
                    // "MIN": result = min(src_0, src_1)
                    else if (current->get_name() == "ALU-MIN") {
                        result = (src_0 < src_1) ? src_0 : src_1;
                    }
                    // "ADD": result = src_0 + src_1 (residual connection)
                    else if (current->get_name() == "ALU-ADD") {
                        result = src_0 + src_1;
                    }
                    // "SHR": result = src_0 >> src_1 (fixed-point scaling)
                    else {
                        result = src_0 >> src_1;
                    }

                    // STEP 9: write result back to acc_mem (destination)
                    acc_mem[dst_idx][b][oc] = result;

                    // STEP 10: also write to out_mem (truncated to 8 bits for Store)
                    out_mem[dst_idx][b][oc] = (int8_t)(result & 0xFF);
                }
            }
        }
    }
} // end micro-op loop

```

```
// STEP 11: advance inner offsets
dst_offset_in += current->get_alu_inner_loop_dst();
src_offset_in += current->get_alu_inner_loop_src();

} // end inner loop

// STEP 12: advance outer offsets
dst_offset_out += current->get_alu_outer_loop_dst();
src_offset_out += current->get_alu_outer_loop_src();

} // end outer loop
```

7. LOAD ACC — Bias Loading into acc_mem

When Compute receives a LOAD instruction where `memory_type = ACC` (accumulator), it loads bias values from DRAM into `acc_mem`. This is the third case inside `dependencies_received()`:

```
// — LOAD ACC: Load bias from "DRAM" into acc_mem —————
// In HLS: uses load_pad_2d() to DMA from biases[] DRAM pointer into acc_mem
// In SystemC: the "DRAM" is simulated — we typically zero-initialise or use
// a fixed bias array. The key loop structure is:

// get_y_size() = number of rows to load
// get_x_size() = number of cols per row
// (sram_base from instruction = where in acc_mem to start writing)

int sram_idx = 0; // in real implementation: from instruction sram_base field

for (int y = 0; y < current->get_y_size(); y++) {
    for (int x = 0; x < current->get_x_size(); x++) {
        // In HLS: acc_mem gets data from DMA; in SystemC: zero or bias value
        for (int b = 0; b < 1; b++) {
            for (int oc = 0; oc < 16; oc++) {
                acc_mem[sram_idx + x][b][oc] = 0; // or real bias value
            }
        }
    }
    sram_idx += current->get_x_size();
}
```

`get_y_size()` and `get_x_size()` are available in the Instruction class. In the current simulator these are already read during parsing — you just call `current->get_y_size()`.

8. Complete dependencies_received() — Putting It All Together

```
void ComputeModule::dependencies_received() {
    std::string name = current->get_name();

    if (name == "GEMM") {
        int dst_offset_out = 0, src_offset_out = 0, wgt_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
            int dst_offset_in = dst_offset_out;
            int src_offset_in = src_offset_out;
            int wgt_offset_in = wgt_offset_out;

            for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
                for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                    uint32_t uop = uop_mem[upc];
                    int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in;
                    int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in;
```

```

        int wgt_idx = (int)((uop >> 22) & 0x3FF) + wgt_offset_in;

        for (int b = 0; b < 1; b++) {
            for (int oc = 0; oc < 16; oc++) {
                int32_t accum = acc_mem[dst_idx][b][oc];
                int32_t tmp = 0;
                for (int ic = 0; ic < 16; ic++) {
                    tmp += (int32_t)(inp_mem[src_idx][b][ic] * wgt_mem[wgt_idx][oc][ic]);
                }
                accum += tmp;
                acc_mem[dst_idx][b][oc] = current->get_reset_out() ? 0 : accum;
                out_mem[dst_idx][b][oc] = (int8_t)(accum & 0xFF);
            }
        }
        dst_offset_in += current->get_gemm_inner_loop_acc();
        src_offset_in += current->get_gemm_inner_loop_inp();
        wgt_offset_in += current->get_gemm_inner_loop_wgt();
    }
    dst_offset_out += current->get_gemm_outer_loop_acc();
    src_offset_out += current->get_gemm_outer_loop_inp();
    wgt_offset_out += current->get_gemm_outer_loop_wgt();
}

}
else if (name.rfind("ALU", 0) == 0 || name == "ALU") {
    int dst_offset_out = 0, src_offset_out = 0;

    for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
        int dst_offset_in = dst_offset_out;
        int src_offset_in = src_offset_out;

        for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {
            for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                uint32_t uop = uop_mem[upc];
                int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in;
                int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in;

                for (int b = 0; b < 1; b++) {
                    for (int oc = 0; oc < 16; oc++) {
                        int32_t src_0 = acc_mem[dst_idx][b][oc];
                        int32_t src_1 = acc_mem[src_idx][b][oc];
                        int32_t result;
                        if (name == "ALU-MAX" || name == "ALU")
                            result = (src_0 > src_1) ? src_0 : src_1; // ReLU-Like
                        else if (name == "ALU-MIN")
                            result = (src_0 < src_1) ? src_0 : src_1;
                        else if (name == "ALU-ADD")
                            result = src_0 + src_1;
                        else
                            result = src_0 >> src_1; // SHR
                        acc_mem[dst_idx][b][oc] = result;
                        out_mem[dst_idx][b][oc] = (int8_t)(result & 0xFF);
                    }
                }
            }
        }
        dst_offset_in += current->get_alu_inner_loop_dst();
    }
}

```

```
        src_offset_in += current->get_alu_inner_loop_src();
    }
    dst_offset_out += current->get_alu_outer_loop_dst();
    src_offset_out += current->get_alu_outer_loop_src();
}
}
else if (name == "LOAD ACC") {
    int sram_idx = 0;
    for (int y = 0; y < current->get_y_size(); y++) {
        for (int x = 0; x < current->get_x_size(); x++) {
            for (int b = 0; b < 1; b++)
                for (int oc = 0; oc < 16; oc++)
                    acc_mem[sram_idx + x][b][oc] = 0; // zero-init bias
        }
        sram_idx += current->get_x_size();
    }
}
// NOP / FINISH: no computation - fall through

finish.notify(latency()); // always last: advance simulation time
}
```

9. Worked Example: GEMM with 4 micro-ops, 3×2 loops

Instruction: GEMM | pc=42 | pop_prev=1 | push_next=1 | range_0=0 | range_1=4 | outer_loop_iter=3 | inner_loop_iter=2 | all gemm_*_loop_* strides = 1 | reset_out=false

Loop structure

Level	Variable	Runs	Counts
Outer	it_out	0, 1, 2	3 iterations
Inner	it_in	0, 1	2 iterations per outer
Micro-op	upc	0, 1, 2, 3	4 micro-ops per inner
Tensor B	b	0	1 (BATCH=1)
Tensor OC	oc	0..15	16 output channels
Tensor IC	ic	0..15	16 input channels (dot product)

Total MAC operations: $3 \times 2 \times 4 \times 1 \times 16 \times 16 = 6144$ MAC (multiply-accumulate).

Trace of offsets through the loops

it_out	it_in	upc	dst_offset_in	src_offset_in	wgt_offset_in	dst_idx (if uop[10:0]=0)
0	0	0	0	0	0	0+0=0
0	0	1	0	0	0	uop[10:0]+0
0	0	2	0	0	0	uop[10:0]+0
0	0	3	0	0	0	uop[10:0]+0
0	1	0..3	1	1	1	uop[10:0]+1 (offset by 1)
1	0	0..3	1	1	1	uop[10:0]+1
1	1	0..3	2	2	2	uop[10:0]+2
2	0	0..3	2	2	2	uop[10:0]+2
2	1	0..3	3	3	3	uop[10:0]+3

Each increase in offset selects the next SRAM tile — so the loops systematically walk through different slices of inp_mem, wgt_mem, and acc_mem.

After all loops complete

acc_mem now contains updated accumulator values. out_mem contains the 8-bit truncated results.

`finish.notify(latency())` then fires with:

```
latency() = (range_1 - range_0) * outer_loop_iter * inner_loop_iter + 100  
          = (4 - 0) * 3 * 2 + 100  
          = 24 + 100 = 124 ns
```

After 124 simulated ns, `finalize_instruction()` runs and `push_next=1` → c2s token is sent to Store, telling it the results are ready in `out_mem`.

10. Quick Reference — Everything You Need

What you want to know	Where to find it in the code
What type of instruction is this?	<code>current->get_name()</code> → "GEMM", "ALU", "LOAD ACC", "NOP-COMPUTE-STAGE", "FINISH"
Outer loop count	<code>current->get_outer_loop_iter()</code>
Inner loop count	<code>current->get_inner_loop_iter()</code>
Micro-op range	<code>current->get_range_0()</code> (start) to <code>current->get_range_1()</code> (end, exclusive)
GEMM outer strides (acc, inp, wgt)	<code>get_gemm_outer_loop_acc/inp/wgt()</code>
GEMM inner strides (acc, inp, wgt)	<code>get_gemm_inner_loop_acc/inp/wgt()</code>
ALU outer strides (dst, src)	<code>get_alu_outer_loop_dst/src()</code>
ALU inner strides (dst, src)	<code>get_alu_inner_loop_dst/src()</code>
Should we zero acc before writing?	<code>current->get_reset_out()</code> → true = write 0
LOAD ACC tile dimensions	<code>current->get_y_size()</code> × <code>current->get_x_size()</code>
Decode GEMM micro-op dst/src/wgt	<code>(uop>>0)&0x7FF</code> , <code>(uop>>11)&0x7FF</code> , <code>(uop>>22)&0x3FF</code>
Decode ALU micro-op dst/src	<code>(uop>>0)&0x7FF</code> , <code>(uop>>11)&0x7FF</code>
Latency formula — GEMM	$(\text{range_1} - \text{range_0}) \times \text{outer} \times \text{inner} + 100 \text{ ns}$
Latency formula — ALU	$2 \times (\text{range_1} - \text{range_0}) \times \text{outer} \times \text{inner} + 100 \text{ ns}$